Ablation Study Report

Author: Rod (Ruoding) Tian — @ruodingt

Goal: Minimise validation loss on Hacker News titles (100k, 7 epochs, seed=1337).

Final best: 28L×256d×10240 + Muon+AdamW + WSD + RoPE + SwigLU + tie_weights + token_anchor → **val_loss = 1.1650 - 1.1660**

The best result (with val_loss: 1.165-1.166) comes from **g12_00** in group 12 baseline.

Key drivers behind performance (in terms of val loss) comes from:

- Muon optimiser
- Vocab size
- deeper but narrower models

We have 12 groups of ablation study which tells the entire journey of pushing down val_loss.

In terms of training throughput, we used bf16 AMP and torch compile to speed up the training on AMD GPU.

Environment

Hardware

System	AMD Ryzen AI MAX+ 395 (APU — unified CPU/GPU memory)
GPU	AMD Radeon 8060S — gfx1151 (RDNA4), 40 CUs, 2900 MHz
Memory	128 GB unified memory — BIOS split: ~64 GB GPU / ~62 GB CPU
CPU	AMD Ryzen AI MAX+ 395, 16-core, 5187 MHz boost

Software

PyTorch	2.9.1+rocm7.1.1
ROCm	7.1
Attention kernel	FlashAttention-2 (via ROCm port)
Precision	bfloat16
Compilation	`torch.compile` enabled

Training Setup

Dataset	Hacker News post titles — `julien040/hacker-news-posts`
Train / val split	90k / 10k titles (val_frac=0.10)
Epochs	7 (fixed)

Seed	1337 (fixed)
Batch size	64 sequences × 128 tokens = 8192 tokens/batch
Tokeniser	BPE, vocab_size (8k / 10k (best) / 12k / 16k)

Hyperparameter Reference

Architecture

Param	Baseline	Final best	Description
`layer_pattern`	`"A"*6`	`"A"*28`	Sequence of layer types: `"A"`=Attention, `"M"`=Mamba. Length = `n_layer`. Pure transformer = `"A"*N`.
`d_model`	512	256	Residual stream width (hidden dimension).
`n_q_head` / `n_kv_heads`	8 / 8	4 / 4	Query / key-value head counts. `n_kv_heads=n_q_head` → MHA; `n_kv_heads=1` → MQA; in between → GQA.
`vocab_size`	16000	10240	BPE vocabulary size (8k or 16k).
`mlp_act`	`gelu`	`swiglu`	MLP activation: `gelu` (baseline), `relu_sq` (ReLU²), `swiglu` (SwiGLU with gated projection).
`tie_weights`	`False`	`True`	Share token embedding and lm_head weight matrices. Reduces params by `vocab_size × d_model`.
`pos_emb`	`learned`	`rope`	Positional encoding: `learned` (absolute, per-position) or `rope` (Rotary Position Embedding, applied to Q and K).
`norm`	`layernorm`	`layernorm`	Normalisation layer: `layernorm` (with bias) or `rmsnorm` (no bias, no mean-centering).
`logit_softcap`	`0.0`	`0.0`	Gemma-style soft-capping of final logits: `logit = cap * tanh(logit / cap)`. `0.0` = disabled.
`use_token_anchor`	`False`	`True`	At each layer, add a learned linear combination of the original token embedding x_0 to the residual stream: `x_l += λ_l * x_0`. Provides a depth-invariant shortcut.
`use_resid_scale`	`False`	`False`	Per-layer learnable scalar on the residual branch (requires `use_token_anchor`).
`use_rezero`	`False`	`False`	Replace residual `x + F(x)` with `x + α * F(x)`, where α is a learnable scalar initialised to 0. Intended to improve training stability at initialisation.
`weight_init`	`gpt2`	`gpt2`	Parameter initialisation scheme: `gpt2` (std scaled by `1/√(2·n_layer)` for residual projections) or `muon_uniform` (uniform ±1/√fan_in, suited for Muon).

Skip Connections (Group 9)

Param	Baseline	Final best	Description
`use_value_residual`	`False`	`False`	Add current-layer input to value projections before attention: `v += x` (reshaped to head dims). Requires MHA (`n_kv_heads == n_q_head`).
`use_value_residual_x0`	`False`	`False`	Add original token embedding to value projections: `v += x₀`. Injects raw token identity deep into attention values.
`use_value_carry`	`False`	`False`	Cross-layer value carry: `v_l = Wv(x) + λ * v_{l−1}`, where `λ` is a learnable scalar initialised to 0. Passes the previous layer's value tensor forward.

Optimiser

Param	Baseline	Final best	Description
`optimizer_type`	`sgd`	`muon_adamw`	`muon_adamw`: Muon for 2-D weight matrices (attention, MLP projections) + AdamW for all other params (norms, biases, embeddings). `sgd`: vanilla SGD baseline.
`muon_lr`	—	`0.02`	Learning rate for Muon. Muon is a second-order-inspired optimiser that orthogonalises the gradient in weight-matrix space using Newton-Schulz iterations.
`adamw_lr`	—	`0.0003`	AdamW learning rate for 1-D parameters (norm scales, biases).
`emb_lr`	—	`0.003`	AdamW learning rate for token embeddings (slightly higher than `adamw_lr` because embedding rows are updated sparsely).
`lr_schedule`	`cosine`	`wsd`	`wsd` (Warmup–Stable–Decay): linear warmup → flat at peak LR → cosine decay. `cosine`: standard cosine annealing from peak to `min_lr`.
`warmup_frac`	`0.0`	`0.05`	Fraction of total training steps used for linear LR warmup.
`decay_frac`	—	`0.6`	(WSD only) Fraction of total steps devoted to the final cosine decay phase.
`min_lr_frac`	`0.0`	`0.05`	LR decay floor as a fraction of peak LR. `0.0` decays all the way to zero; `0.05` stops at 5% of peak.
`adamw_wd`	—	`0.1`	AdamW weight decay (L2 regularisation coefficient).

Group 1 — Optimizer & Schedule

Key finding: Muon+AdamW with RoPE+WSD gives the biggest single jump (−0.54). gpt2 init beats muon_uniform init.

Exp	Config delta	val_loss	Δ	Params	tok/s
g1_00	—	1.7319	+0.0000	35.35M	66,340
g1_01	optimizer_type=muon_adamw	1.2333	-0.4986	35.35M	53,642
g1_02	optimizer_type=muon_adamw, weight_init=muon_uniform	1.2781	-0.4538	35.35M	58,560
g1_03	optimizer_type=muon_adamw, use_rezero=true	1.2576	-0.4743	35.35M	54,824
g1_04	optimizer_type=muon_adamw, weight_init=muon_uniform, use_rezero=true	1.2964	-0.4355	35.35M	58,813
g1_05	optimizer_type=muon_adamw, weight_init=muon_uniform, pos_emb=rope	1.2562	-0.4757	35.29M	54,705
g1_06	optimizer_type=muon_adamw, clip_norm_mode=adamw	1.2362	-0.4957	35.35M	55,947
g1_07	optimizer_type=muon_adamw, weight_init=muon_uniform, clip_norm_mode=adamw	1.2797	-0.4522	35.35M	58,159
g1_08	optimizer_type=muon_adamw, pos_emb=rope	1.2119	-0.5200	35.29M	57,054
g1_09	optimizer_type=muon_adamw, lr_schedule=wsd, warmup_frac=0.05, min_lr_frac=0.05	1.2182	-0.5137	35.35M	58,214
g1_10	optimizer_type=muon_adamw, lr_schedule=wsd, warmup_frac=0.05, min_lr_frac=0.05, weight_init=muon_uniform	1.2503	-0.4816	35.35M	57,801
g1_11	**optimizer_type=muon_adamw, pos_emb=rope, lr_schedule=wsd, warmup_frac=0.05, min_lr_frac=0.05 ✓**	**1.1953**	**-0.5366**	**35.29M**	**57,342**
g1_12	optimizer_type=muon_adamw, pos_emb=rope, lr_schedule=wsd, warmup_frac=0.05, min_lr_frac=0.05, weight_init=muon_uniform	1.2261	-0.5058	35.29M	57,142

Note: baseline 1.7319 rather than 1.754 is most likely due to 2 reasons:

1. no dropout in position embedding in the implementation
2. Head embedding is untied from the input token embedding

Group 2 — Activation & Architecture Tricks (6L×512d)

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
```

Key finding: SwigLU + tie_weights = best combo (−0.012). token_anchor, softcap, MQA all neutral at 6L.

Exp	Config delta	val_loss	Δ	Params	tok/s
g2_00	—	1.1951	-0.5368	35.29M	57,781
g2_01	use_token_anchor=true	1.1954	+0.0003	35.29M	53,274
g2_02	logit_softcap=15	1.1955	+0.0004	35.29M	54,392
g2_03	norm=rmsnorm	1.1954	+0.0003	35.28M	50,872
g2_04	n_kv_heads=1	1.1990	+0.0039	32.53M	50,306
g2_05	tie_weights=true	1.1925	-0.0026	27.1M	54,788
g2_06	use_token_anchor=true, logit_softcap=15	1.1956	+0.0005	35.29M	53,512
g2_07	mlp_act=swiglu	1.1872	-0.0079	35.08M	53,560
g2_08	mlp_act=relu_sq	1.1878	-0.0073	35.29M	55,220
g2_09	**mlp_act=swiglu, tie_weights=true ✓**	**1.1836**	**-0.0115**	**26.88M**	**56,363**
g2_10	mlp_act=relu_sq, tie_weights=true	1.1894	-0.0057	27.1M	55,970

Group 3 — Confirm Best Config (6L×512d)

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
mlp_act: swiglu
tie_weights: true
```

Key finding: All tricks neutral or slightly negative at shallow depth. Best config = base.

Exp	Config delta	val_loss	Δ	Params	tok/s
g3_00	**— ✓**	**1.1843**	**-0.5476**	**26.88M**	**59,773**
g3_01	norm=rmsnorm	1.1862	+0.0019	26.88M	53,631

g3_02	use_token_anchor=true	1.1858	+0.0015	26.88M	55,249
g3_03	logit_softcap=15	1.1860	+0.0017	26.88M	56,073
g3_04	n_kv_heads=1	1.1852	+0.0009	24.13M	54,405
g3_05	use_token_anchor=true, logit_softcap=15	1.1851	+0.0008	26.88M	55,476

Group 4 — Vocab Size & Depth (6L)

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
mlp_act: swiglu
tie_weights: true
```

Key finding: vocab=16k is optimal; 8k hurts. Adding layers at 512d doesn't help — motivates architecture search.

Exp	Config delta	val_loss	Δ	Params	tok/s
g4_00	**— ✓**	**1.1838**	**-0.5481**	**26.88M**	**59,973**
g4_01	vocab_size=8192	1.2100	+0.0262	22.89M	63,649
g4_02	12L	1.1863	+0.0025	45.57M	31,516
g4_03	vocab_size=8192, 12L	1.2084	+0.0246	41.58M	34,856

Group 5 — Architecture Search (~30M params)

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
mlp_act: swiglu
tie_weights: true
```

The goal at this stage is to find the impact depth vs width of networks. So total number of parameters is capped at ~30m in this group, making it easier to compare.

Key finding: Deeper-narrower consistently wins. 12L×384d = best at 30M budget.

Exp	Config delta	val_loss	Δ	Params	tok/s
g5_00	—	1.1842	-0.5477	26.88M	60,129

g5_01	8L, d_model=448, n_q_head=7, n_kv_heads=7	1.1780	-0.0062	26.68M	48,746
g5_02	9L, d_model=448, n_q_head=7, n_kv_heads=7	1.1772	-0.0070	29.12M	46,288
g5_03	**12L, d_model=384, n_q_head=6, n_kv_heads=6 ✓**	**1.1727**	**-0.0115**	**27.4M**	**46,046**
g5_04	13L, d_model=384, n_q_head=6, n_kv_heads=6	1.1736	-0.0106	29.17M	43,970
g5_05	5L, d_model=576, n_q_head=9, n_kv_heads=9	1.1908	+0.0066	29.14M	51,621
g5_06	4L, d_model=640, n_q_head=10, n_kv_heads=10	1.1978	+0.0136	30.08M	50,299

Group 6 — Architecture Search (~40M params) + GQA

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
mlp_act: swiglu
tie_weights: true
```

Key finding: Deeper-narrower trend continues. GQA does not help. 20L×320d = best.

Exp	Config delta	val_loss	Δ	Params	tok/s
g6_00	—	1.1850	-0.5469	26.88M	60,134
g6_01	14L, d_model=384, n_q_head=6, n_kv_heads=6	1.1725	-0.0125	30.94M	41,639
g6_02	16L, d_model=384, n_q_head=6, n_kv_heads=6	1.1733	-0.0117	34.48M	37,453
g6_03	18L, d_model=384, n_q_head=6, n_kv_heads=6	1.1747	-0.0103	38.02M	33,695
g6_04	19L, d_model=384, n_q_head=6, n_kv_heads=6	1.1757	-0.0093	39.79M	31,970
g6_05	**20L, d_model=320, n_q_head=5, n_kv_heads=5 ✓**	**1.1710**	**-0.0140**	**29.31M**	**36,804**
g6_06	24L, d_model=320, n_q_head=5, n_kv_heads=5	1.1724	-0.0126	34.15M	32,097

g6_07	28L, d_model=320, n_q_head=5, n_kv_heads=5	1.1730	-0.0120	38.99M	28,083
g6_08	16L, d_model=384, n_q_head=6, n_kv_heads=2	1.1731	-0.0119	31.34M	40,352
g6_09	21L, d_model=384, n_q_head=6, n_kv_heads=2	1.1779	-0.0071	39.21M	31,206

Group 7 — Narrow-and-Deep: 256d at Various Depths

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
mlp_act: swiglu
tie_weights: true
```

Base for Δ: best from group6 (g6_05 = 20L×320d, val_loss=1.1710) **Key finding:** 28L×256d slightly edges out 20L×320d. Diminishing returns beyond 28L.

Exp	Config delta	val_loss	Δ	Params	tok/s
g6_05	—	1.1710	-0.5609	29.31M	36,804
g7_01	**28L, d_model=256, n_q_head=4, n_kv_heads=4 ✓**	**1.1708**	**-0.0002**	**26.6M**	**35,661**
g7_02	32L, d_model=256, n_q_head=4, n_kv_heads=4	1.1720	+0.0010	29.82M	32,272
g7_03	36L, d_model=256, n_q_head=4, n_kv_heads=4	1.1736	+0.0026	33.03M	29,143
g7_04	40L, d_model=256, n_q_head=4, n_kv_heads=4	1.1738	+0.0028	36.25M	26,366
g7_05	45L, d_model=256, n_q_head=4, n_kv_heads=4	1.1759	+0.0049	40.27M	23,705

Group 8 — Depth-Dependent Tricks (28L×256d)

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
mlp_act: swiglu
```

```
tie_weights: true
layer_pattern: AAAAAAAAAAAAAAAAAAAAAAAAAAAA
d_model: 256
n_q_head: 4
n_kv_heads: 4
```

Key finding: token_anchor helps at depth (−0.0013). softcap and resid_scale neutral or negative. ReZero mildly negative. Note: token_anchor was neutral at 6L (group3) — it is depth-dependent.

Exp	Config delta	val_loss	Δ	Params	tok/s
g8_00	—	1.1713	-0.5606	26.6M	43,133
g8_01	**use_token_anchor=true ✓**	**1.1700**	**-0.0013**	**26.6M**	**34,954**
g8_02	logit_softcap=15	1.1733	+0.0020	26.6M	37,553
g8_03	use_token_anchor=true, logit_softcap=15	1.1726	+0.0013	26.6M	35,961
g8_04	use_token_anchor=true, use_resid_scale=true	1.1711	-0.0002	26.6M	33,322
g8_05	use_token_anchor=true, use_resid_scale=true, logit_softcap=15	1.1725	+0.0012	26.6M	35,471
g8_06	use_token_anchor=true, use_rezero=true	1.1782	+0.0069	26.6M	34,508
g8_07	use_rezero=true	1.1715	+0.0002	26.6M	36,599

Group 9 — Value Skip Connections (28L×256d)

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
mlp_act: swiglu
tie_weights: true
layer_pattern: AAAAAAAAAAAAAAAAAAAAAAAAAAAA
d_model: 256
n_q_head: 4
n_kv_heads: 4
```

Three variants tested: v += x (current-layer residual), v += x_0 (original embedding), v += λ·v_{l-1} (cross-layer carry). **Key finding:** All value residual variants are ≥ anchor alone. Anchor-only = best.

Exp	Config delta	val_loss	Δ	Params	tok/s
g9_00	—	1.1711	-0.5608	26.6M	42,849
g9_01	use_value_residual=true	1.1738	+0.0027	26.6M	36,573

g9_02	use_value_residual=true, use_token_anchor=true	1.1730	+0.0019	26.6M	34,686
g9_03	use_value_residual=true, logit_softcap=15	1.1748	+0.0037	26.6M	36,489
g9_04	use_value_residual=true, use_token_anchor=true, logit_softcap=15	1.1741	+0.0030	26.6M	35,186
g9_05	use_value_residual_x0=true	1.1710	-0.0001	26.6M	36,065
g9_06	use_value_residual_x0=true, use_token_anchor=true	1.1701	-0.0010	26.6M	34,327
g9_07	use_value_residual=true, use_value_residual_x0=true	1.1730	+0.0019	26.6M	35,470
g9_08	use_value_carry=true	1.1712	+0.0001	26.6M	35,342
g9_09	use_value_carry=true, use_token_anchor=true	1.1706	-0.0005	26.6M	33,349
g9_10	**use_token_anchor=true ✓**	**1.1696**	**-0.0015**	**26.6M**	**41,498**

Group 10 — Vocab Size & Architecture Variants (28L×256d)

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
decay_frac: 0.6
dropout: 0.1
mlp_act: swiglu
tie_weights: true
use_token_anchor: true
layer_pattern: AAAAAAAAAAAAAAAAAAAAAAAAAAAA
d_model: 256
n_q_head: 4
n_kv_heads: 4
```

Key findings:

- vocab=10240 beats 16k (−0.004); 8k and 12k both hurt
- `separate_kv` : splits fused kv_proj into square k_proj+v_proj; +5% tok/s, neutral loss
- `spectral_clip` : −19% tok/s, no loss benefit — dropped
- `muon_attn_only` : routes MLP matrices to AdamW — catastrophic (+0.041); Muon is essential for MLP
- Smaller MLP (hidden=512/384): −27%/−45% params, slight loss increase — compression headroom exists but costs quality

Exp	Config delta	val_loss	Δ	Params	tok/s

g10_00	—	1.1700	-0.5619	26.6M	40,991
g10_01	**vocab_size=10240 ✓**	**1.1660**	**-0.0040**	**25.13M**	**37,430**
g10_02	vocab_size=12288	1.2016	+0.0316	25.65M	36,788
g10_03	vocab_size=8192	1.1937	+0.0237	24.61M	38,085
g10_04	separate_kv=true, vocab_size=10240	1.1668	-0.0032	25.13M	43,205
g10_05	vocab_size=10240, spectral_clip=1	1.1673	-0.0027	25.13M	33,219
g10_06	vocab_size=10240, muon_attn_only=true	1.2106	+0.0406	25.13M	44,997
g10_07	vocab_size=10240, muon_attn_only=true, mlp_lr=0.001	1.2065	+0.0365	25.13M	45,593
g10_08	vocab_size=10240, mlp_expand=3	1.1687	-0.0013	21.0M	40,890
g10_09	vocab_size=10240, mlp_expand=2.25	1.1697	-0.0003	18.25M	43,831

Group 11 — Value Embeddings / E Layers (ResFormer-style)

Dedicated per-layer embedding tables injected into V via a learned per-head gate: `v += 3·σ(Linear(x[:12])) * ve_table(idx)`. Layer type `E` in `layer_pattern` enables this; embedding tables are separate from `token_emb`, optimised with `emb_lr`.

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
decay_frac: 0.6
dropout: 0.1
mlp_act: swiglu
mlp_expand: 4.0
tie_weights: true
use_token_anchor: true
vocab_size: 16000
layer_pattern: AAAAAAAAAAAAAAAAAAAAAAAAAAAA
d_model: 256
n_q_head: 4
n_kv_heads: 4
```

Key finding: Gains are marginal across all VE configs. vocab=10k+VE rows benefit more from the vocab change than from VE itself. gate_channels makes no meaningful difference.

Exp	Config delta	val_loss	Δ	Params	tok/s
g11_00	—	1.1701	-0.5618	26.6M	41,602
g11_01	28L+2E@9,19	1.1699	-0.0002	34.8M	40,605

g11_02	28L+3E@9,18,27	1.1695	-0.0006	38.89M	40,329
g11_03	28L+2E@9,19, ve_gate_channels=16	1.1702	+0.0001	34.8M	40,526
g11_04	28L+3E@9,18,27, vocab_size=10240	1.1662	-0.0039	32.99M	42,169
g11_05	28L+3E@9,18,27, vocab_size=8192	1.1933	+0.0232	30.9M	42,906
g11_06	28L+4E@3,11,19,27	1.1696	-0.0005	42.99M	39,742
g11_07	28L+2E@13,27	1.1694	-0.0007	34.8M	40,456
g11_08	vocab_size=10240, 28L	1.1666	-0.0035	25.13M	42,842
g11_9	vocab_size=10240, 28L+5E@7,12,17,22,27	1.1661	-0.0040	38.24M	41,682
g11_10	vocab_size=10240, 28L+4E@3,11,19,27	1.1654	-0.0047	35.62M	41,728
g11_11	**vocab_size=10240, 28L+4E@3,11,19,27, muon_lr=0.025 ✓**	**1.1651**	**-0.0050**	**35.62M**	**41,772**

Group 12 — Mamba Hybrid (first layer)

```
optimizer_type: muon_adamw
pos_emb: rope
lr_schedule: wsd
warmup_frac: 0.05
min_lr_frac: 0.05
decay_frac: 0.6
dropout: 0.1
mlp_act: swiglu
mlp_expand: 4.0
tie_weights: true
use_token_anchor: true
vocab_size: 10240
d_model: 256
n_q_head: 4
n_kv_heads: 4
muon_lr: 0.02
```

Key finding: Replacing the first attention layer with Mamba SSM hurts both quality (+0.0076) and throughput (−42% tok/s). Pure attention remains better at this scale and sequence length.

Exp	Config delta	val_loss	Δ	Params	tok/s
g12_00	**28L+4E@3,11,19,27 ✓**	**1.1650**	**-0.5669**	**35.62M**	**41,947**
g12_01	28L+1M@0+4E@3,11,19,27	1.1726	+0.0076	35.82M	24,174

Observation: With Mamba replacing the first layer, the loss drops faster at early epoches, but it converges at higher val loss in late epoches.

Appendix — GPU Profiling

Profiled on AMD 8060S using `torch.profiler` (CPU + GPU activities). 10 steps after 3 warmup steps, with `torch.compile` active.

Config: `AAAEAAAAAAAEAAAAAAAEAAAAAAAE` · 35.6M params · 28L × 256d · vocab=10240
Date: 2026-05-28

Headline Results

```
============================================================
GPU UTILISATION
============================================================
  Time span:     1687.4 ms
  Compute:       1582.5 ms  (93.8%)
  Void:           104.9 ms  (6.2%)

============================================================
VOID DISTRIBUTION
============================================================
  [    0 – 2     µs]:  5247 voids      9.9 ms
  [    2 – 5     µs]:  7939 voids     16.9 ms
  [    5 – 10    µs]:  5877 voids     38.8 ms
  [   10 – 20    µs]:   139 voids      1.6 ms
  [   20 – 50    µs]:     3 voids      0.1 ms
  [   50 – 100   µs]:     2 voids      0.1 ms
  [  100 – 500   µs]:     3 voids      0.4 ms
  [  500 – 1000  µs]:    35 voids     19.6 ms
  [ 1000 – 5000  µs]:    14 voids     17.5 ms
  [ 5000 – ∞     µs]:     0 voids      0.0 ms

  Launch overhead (≤200µs): 67.8 ms  (19210 voids)
  Bubbles        (>200µs): 37.1 ms  (49 voids)
```

Source trace file can be found in profile_out.

Some initial analysis can be found in profiling.md.

The remaining idle time is dominated by kernel launch overhead (~68ms). `torch.compile` reduces this by fusing operators into fewer kernels, but per-kernel HIP dispatch cost remains. Eliminating it entirely would require HIP Graphs (`mode="reduce-overhead"`), which ROCm supports but was not tested.

Overall, the potential gain from further optimisation is limited.

Appendix — Custom Kernel Benchmarks

Hardware: AMD Ryzen AI MAX+ 395, gfx1151 (RDNA4), 40 CU, ~300 GB/s unified memory.

RMSNorm (Triton)

Benchmark shape: `(128, 64, 384)` bfloat16.

Implementation	Speed
`nn.RMSNorm` (91.3 µs)	1.0x
Triton + autotune (49.7 µs)	**1.84x**

autotune selected: fwd `num_warps=2` , bwd `num_warps=1` . **Status: Not used in final config** — final architecture uses `layernorm` (ablation g3_01 showed neutral vs rmsnorm).

RoPE (Triton)

Benchmark shape: `B=64, H=4, T=128, D=64` (actual training config), bfloat16 fwd+bwd.

Implementation	Speed (fwd+bwd)
Native PyTorch (283.4 µs)	1.0x
Triton (246.9 µs)	**1.15x**

Status: Not enabled — discovered faster only after correcting benchmark shape late in the project. Correctness verified; enabling requires one line change in `rope.py` .

Fused Linear + Cross-Entropy (v2)

True kernel fusion: matmul written inside the Triton kernel; `[BT, V]` logit tensor never materialised. Two-pass algorithm: (1) online softmax + collect target logit; (2) recompute logits → grad_x + grad_W.

Full training step benchmark (B=64, T=128, V=10240, d=256, L=28):

Implementation	ms/step	Peak Memory
Standard CE	1619 ms (1.0x)	7109 MB
Fused CE v2, autotuned	**1559 ms (1.04x)**	**5815 MB (1.22x savings)**

Note: full-step times measured without `torch.compile` or operator fusion — reference only, not representative of compiled training performance. CE kernel contribution is diluted by all other ops. **Status: Optional** (`use_fused_ce=True`). Primary value is memory savings (1.22x). Further testing needed to understand performance gains on other AMD GPUs (e.g. MI355X).

Summary — Best Config Evolution

Step	Change	val_loss	Δ
Baseline	SGD + cosine + learned_pos + GELU	1.7319	--
Group 1	→ Muon+AdamW + RoPE + WSD	1.1953	−0.5366
Group 2	→ SwigLU + tie_weights	1.1836	−0.0117
Groups 5-7	→ 28L×256d (deep-narrow arch)	1.1708	−0.0128

Group 8	→ token_anchor	1.1700	−0.0008
Group 9	→ confirmed anchor-only best	1.1696	--
Group 10	→ vocab=10240	1.1660	−0.0040
Group 11	→ 4E value embeddings (pos 3,11,19,27)	**1.1650**	-0.0010 (no significant advantage)
Group 12	No change - Mamba works worse		--

Total improvement: −0.5669 (32.7% relative reduction from baseline)

Reflections

1. Weight initialisation exploration was insufficient

Tested `muon_uniform` vs `gpt2` init in a single group-1 experiment and found `gpt2` wins by 0.03 val_loss. Did not investigate why.

2. Custom kernels before profiling — wrong order

Wrote three Triton kernels (RMSNorm, RoPE, fused CE) before running any profiler. When we eventually profiled the best config with torch profiler traces, GPU utilisation measured from the Chrome Trace was high (>93%) — meaning there was no large dispatch bubble to fix. The correct workflow is: **profile first, identify hot kernels, then write targeted replacements**. Of the three kernels, none ended up in the final training path: RMSNorm is unused (final config uses LayerNorm), RoPE Triton was discovered to be faster only after correcting the benchmark shape late in the project, and fused CE provides memory savings but marginal speed improvement on gfx1151.

3. Vocab size was fixed too early

`vocab_size` was not systematically swept until Group 10 — after nine groups of experiments all run at `vocab_size=16000` . The final optimal value turned out to be 10240 (−0.004 vs 16k). This means Groups 1–9 were optimising on a suboptimal vocabulary, and some conclusions may not fully transfer: in particular, the value embedding experiments (Group 11) showed VE benefits more with vocab=16k than 10k, suggesting the Group 11 results are partly an artefact of the vocab choice. Vocab size interacts with embedding dimensionality and weight tying; it should be treated as a foundational hyperparameter and swept in the first group rather than the tenth. Interestingly, I did run an analysis on vocab size and identified the elbow point is roughly at 8K and assumed that 8K might be the sweet spot: sequence length slightly longer and saved emb params can go to the budget pools for more depth.

4. Sequential ablation search misses interactions; automatic tuning was underused

Each group performed single-variable search on top of the previous group's best config. This greedy sequential strategy cannot discover interactions between hyperparameters — for example, the optimal `muon_lr` for 28L×256d may differ from the value inherited from Group 1 (6L×512d), and the optimal depth for a given vocab size was never jointly optimised. We did implement Optuna TPE hyperparameter search (`hypertune.py`) but used it only for a narrow LR sweep rather than as the primary search strategy. Investing more in automatic tuning earlier — using Optuna to jointly search over depth, width, vocab, and LR — would likely have found better configurations faster and with less manual iteration.

5. TensorBoard integration added limited value

We integrated TensorBoard (loss curves, LR schedules, weight norms) early in the project. In practice, all experiment tracking and comparison was done through JSONL log files parsed by `collect_results.py`. The TensorBoard writer added code complexity, a `SummaryWriter` dependency, and extra I/O on every training step, with minimal return — the ablation tables in this report can simply be derived from mainrun logs. A leaner approach would be structured JSONL logging only, with a simple `collect_results.py` for post-hoc analysis.

6. results can be more statistically robustness

Could take multiple runs for each experiment to get more statistically robust results.
